

Program – 8: Create Kubernetes Pods using YAML Descriptors

Description

This program demonstrates deploying a Node.js application inside a Kubernetes Pod using declarative YAML instead of imperative commands.

Procedure

Step 1: Create Node.js App (server.js)

```
const http = require("http");

const port = 3000;

const server = http.createServer((req, res) => {
  res.end("Hello from Node.js running inside Kubernetes!");
});

server.listen(port);
```

Step 2: Create ConfigMap (nodejs-configmap.yaml)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nodejs-app-config
data:
  server.js: |
    const http = require("http");

    const port = 3000;

    const server = http.createServer((req, res) => {
      res.end("Hello from Node.js running inside Kubernetes!");
    });

    server.listen(port);
```

Step 3: Create Pod (nodejs-pod.yaml)

```
GNU nano 8.3 nodejs-pod.yaml *
apiVersion: v1
kind: Pod
metadata:
  name: nodejs-pod
  labels:
    app: nodejs-app
spec:
  containers:
  - name: nodejs-container
    image: node:18
    workingDir: /usr/src/app
    command: ["node", "server.js"]
    ports:
    - containerPort: 3000
    volumeMounts:
    - name: nodejs-app-volume
      mountPath: /usr/src/app
  volumes:
  - name: nodejs-app-volume
    configMap:
      name: nodejs-app-config
```

Step 4: Apply Files

```
kubectl apply -f nodejs-configmap.yaml
```

```
kubectl apply -f nodejs-pod.yaml
```

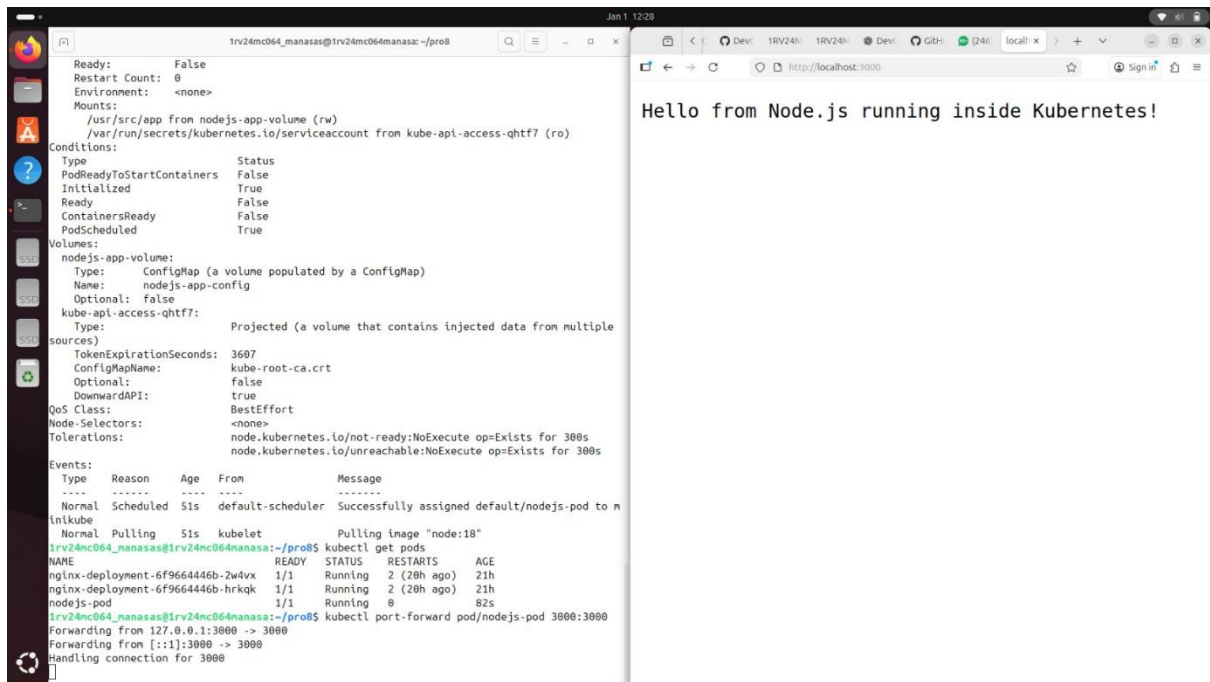
Step 5: Port Forward

```
kubectl port-forward pod/nodejs-pod 3000:3000
```

Open: <http://localhost:3000>

Expected Output

Hello from Node.js running inside Kubernetes!



Result

Successfully deployed a Node.js application inside a Kubernetes Pod using YAML descriptors.